Apuntes · Clase 35 · Bagging y Random Forest

Para leer antes de los ejercicios · lectura de ~45 min

Lo más importante, en 6 ideas

1. **Árbol inestable.** Un árbol profundo cambia mucho con pequeños cambios en los datos: alta varianza.
2. **Bootstrap.** n filas con reemplazo: ≈ 63.2 % únicas y ≈ 36.8 % fuera (OOB, validación interna).
3. **Bagging.** Un árbol por muestra bootstrap; se agrega por voto o promedio. Se entrena en paralelo.
4. **Varianza.** Var = $\rho\sigma^2 + (1-\rho)\sigma^2/B$: más árboles no pueden cruzar el piso $\rho\sigma^2$.
5. **Random Forest.** Bagging + variables al azar en cada nodo (max_features): baja ρ.
6. **Validar, no recitar.** max_features, n_estimators y hojas se eligen con CV u OOB; el test queda para el final.

Cada sección trae una explicación, un **ejemplo resuelto con números distintos** a los de los ejercicios y una **pista** para el notebook `C35_RandomForest.ipynb` o el cuaderno `C35_Ejercicios_Cuaderno_RandomForest.pdf`.

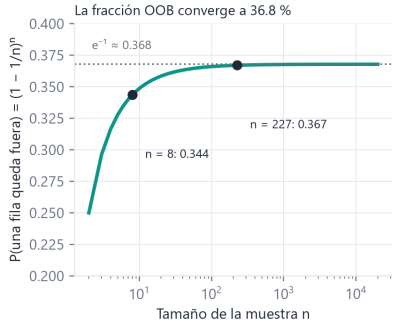
1 Bootstrap y fracción OOB

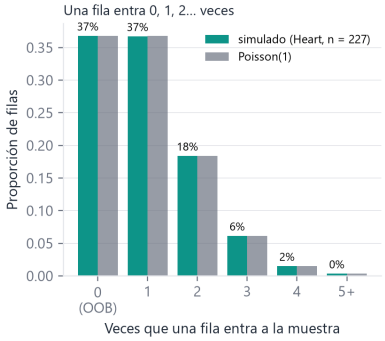
Una muestra bootstrap saca **n filas con reemplazo**. Cada extracción deja fuera a una fila dada con probabilidad 1 − 1/n; tras n extracciones la fila sigue fuera con probabilidad $\mathbf{(1-1/n)^n}$, que converge a $e^{-1} \approx 0.368$. Esas filas son el **OOB** de ese árbol.

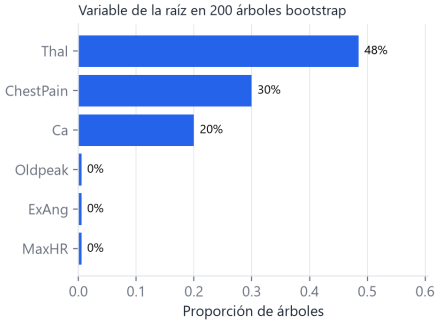
Ejemplo resuelto · cuántas filas quedan fuera

Caso	Cálculo	Resultado
n = 6	$(5/6)^6$	0.335 (el ejemplo de la clase, n = 8: 0.344)
n = 25	$(24/25)^{25} = 0.96^{25}$	0.360
n = 40: filas distintas esperadas	$40 \cdot (1 - 0.975^{40}) = 40 \cdot (1 - 0.363)$	25.5 (63.7 %)
B = 10 árboles: filas sin ninguna predicción OOB	0.632^{10}	1.0 % (con 30 árboles, ≈ 0.0001 %)

Con n pequeño la fracción fuera es algo menor que 36.8 %, pero converge muy rápido: con n = 227 (Heart) es 36.7 %. Por eso hacen falta ≈ 30 árboles para que casi toda fila tenga alguna predicción OOB.







Pista para la Parte A del notebook y el Bloque 1 del cuaderno

Escribe una muestra bootstrap de 8 números, tacha los que salieron y anota el OOB. Luego simula 2.000 muestras y compara con $(1 - 1/n)^n$. Recuerda: la muestra tiene **n** extracciones, no una fracción de n.

2 Bagging: voto y promedio

Bagging entrena un árbol por muestra bootstrap y agrega: en **clasificación**, la moda de los votos o el promedio de probabilidades; en **regresión**, el promedio. Los árboles no se comunican entre sí, así que se pueden entrenar en paralelo.

Ejemplo resuelto · agregar cinco árboles

Árbol	1	2	3	4	5	Agregación
P(Sí)	0.80	0.30	0.70	0.45	0.20	promedio = 0.49
Voto (P > 0.5)	Sí	No	Sí	No	No	2 Sí de 5 → 0.40
Precio (miles USD)	180	210	195	240	175	promedio = 200 (mediana 195)

Con umbral 0.5 ambas reglas dicen No; con un umbral 0.45 la probabilidad media (0.49) dice **Sí** y el voto (0.40) dice No: no son lo mismo. scikit-learn promedia probabilidades; coincide con el voto solo si las hojas son puras (`min_samples_leaf=1`).

Pista para la parte A3 del notebook y el Bloque 3 del cuaderno

En el notebook programas el bagging desde cero (bootstrap + árboles + promedio + OOB a mano) y lo comparas con `BaggingClassifier`.

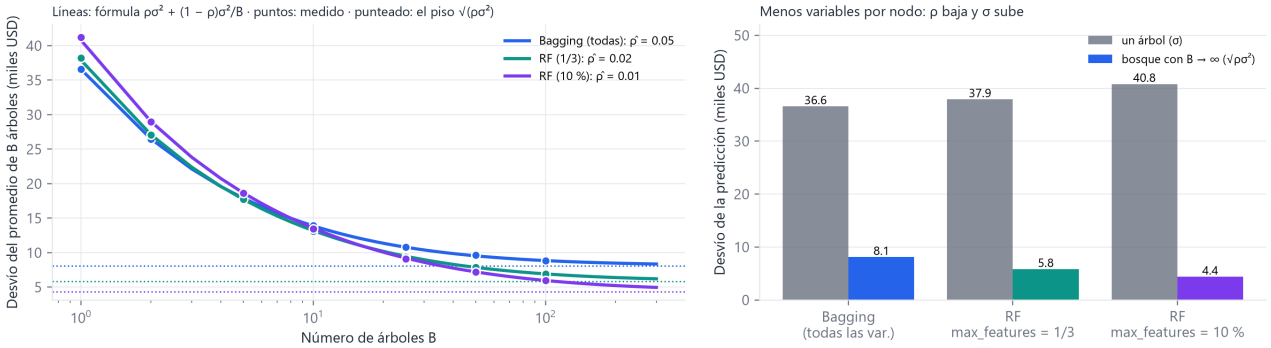
3 Por qué promediar reduce la varianza

Si cada árbol tiene varianza σ^2 y dos árboles cualesquiera tienen correlación ρ, el promedio de B árboles tiene **Var = $\rho\sigma^2 + (1-\rho)\sigma^2/B$**. Con B grande el segundo término desaparece, pero el primero queda: el **piso** $\rho\sigma^2$.

Ejemplo resuelto · $\sigma^2 = 64$ con dos correlaciones

B	$\rho = 0.5$	$\rho = 0.1$
1	32 + 32 = 64	6.4 + 57.6 = 64
4	32 + 8 = 40	6.4 + 14.4 = 20.8
16	32 + 2 = 34	6.4 + 3.6 = 10
∞	32 (desvío 5.66)	6.4 (desvío 2.53)

Con $\rho = 0.5$ pasar de 16 a infinitos árboles solo baja de 34 a 32; con $\rho = 0.1$ el piso es cinco veces menor. Árboles que se parecen ya no aportan: hay que bajar ρ, no solo subir B.



En Ames se midieron σ y ρ repitiendo el entrenamiento con distintas submuestras: $\hat{\rho} = 0.05$ en el bagging y 0.02 en un Random Forest, y las varianzas medidas coinciden con la fórmula. La ρ de la fórmula es la correlación entre las **predicciones** de los árboles (no «de los errores»).

Pista para la parte B2 del notebook y el Bloque 4 del cuaderno

Usa la fórmula con papel antes de abrir el notebook; después verifica con la estimación de ρ y σ. El piso $\sqrt{\rho\sigma^2}$ te dice cuántos árboles son «suficientes».

4 Random Forest y max_features

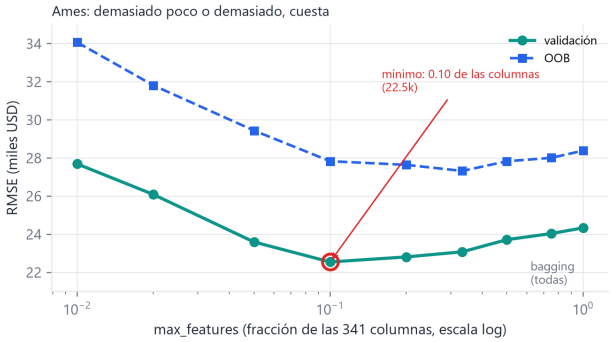
Random Forest añade **variables al azar en cada nodo**: de las p disponibles solo se evalúan m = `max_features`, y el mejor corte se busca ahí. Una variable dominante no puede controlar todos los árboles: se parecen menos y ρ baja. A cambio cada árbol es algo peor. Con m = p el Random Forest **es** un bagging.

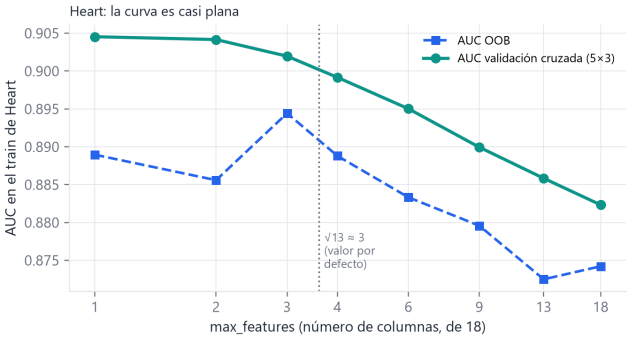
Ejemplo resuelto · una variable dominante

p = 10 variables, m = 4 candidatas por nodo (el valor por defecto √10 daría 3).

Pregunta	Cálculo	Resultado
P(la dominante es candidata en un nodo)	m/p = 4/10	0.40
P(NO es candidata en la raíz)	1 − 0.4	0.60
P(ausente en las raíces de 3 árboles)	0.6^3	0.216
P(ausente en las raíces de 10 árboles)	0.6^{10}	0.006

En el 60 % de los nodos la mejor variable ni siquiera compite: los demás árboles aprenden de otras variables y se vuelven distintos.





Por nodo, no por árbol. Si las variables se sortearan una sola vez por árbol (`BaggingRegressor(max_features=…)`), un árbol con variables malas sería malo en todo su recorrido: en Ames, con el 10 % de las columnas, eso da un RMSE de 27.1k contra 22.5k del bosque que sortea por nodo. Y el valor óptimo se valida: en Ames fue ≈ 10 %; en Heart la curva es casi plana.

Pista para la Parte B del notebook y los Bloques 5 y 6 del cuaderno

En B3 compara «por árbol» y «por nodo» con la misma fracción. Luego haz tu propio barrido de `max_features` y mira la brecha entre validación y OOB.

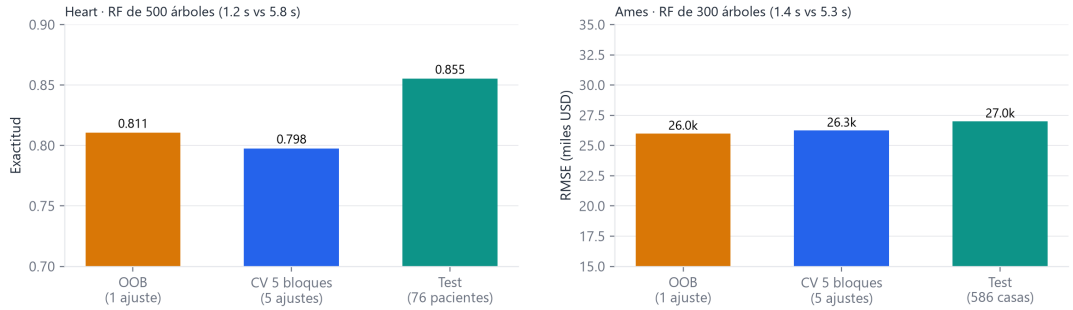
5 OOB, número de árboles y búsqueda de hiperparámetros

El **OOB** predice cada fila solo con los árboles que no la vieron: es una validación interna con **un solo ajuste**. Sirve para comparar configuraciones, pero el informe final necesita un test intacto. Más árboles estabilizan sin sobreajustar y cuestan tiempo; los demás hiperparámetros (max_features, min_samples_leaf, max_depth) se buscan con `GridSearchCV` o `RandomizedSearchCV`.

Ejemplo resuelto · costo de una búsqueda y del paralelismo

Pregunta	Cálculo	Resultado
Ajustes de una grilla 3 × 2 × 2 × 4 con 5 bloques	48 combinaciones × 5	240 ajustes
Bosque de 400 árboles a 0.5 s por árbol, 8 núcleos: tiempo ideal	400 · 0.5 / 8	25 s
Si en la práctica tarda 35 s: aceleración y eficiencia	200 / 35 · (5.7 / 8)	5.7× · 71 %
Árboles en los que cada fila es OOB (B = 200)	0.368 · 200	≈ 74

«Se tarda B/N» es el límite ideal: la aceleración real queda bajo el número de núcleos (en Ames, 6.6× con 16 núcleos lógicos).



OOB, validación cruzada y test dan estimaciones en el mismo rango (Heart 0.811 · 0.798 · 0.855; Ames 26.0k · 26.3k · 27.0k), y el OOB cuesta una quinta parte. Y la búsqueda ayuda poco: en Heart pasó de AUC 0.899 (RF base) a ≈ 0.911 con Grid o Random, por debajo del desvío (≈ 0.03). El profesor lo resumió: la optimización «puede subir unos puntos», no convertir un modelo de 0.70 en uno de 0.95; y se justifica con CV y test, no por el nombre del algoritmo.

Pista para la Parte C del notebook y el Bloque 7 del cuaderno

En C1 calcula el OOB a mano con `estimators_samples_`. En C3 incluye tu configuración base en la grilla y mira el test **una sola vez**.

6 Importancia: impureza y permutación

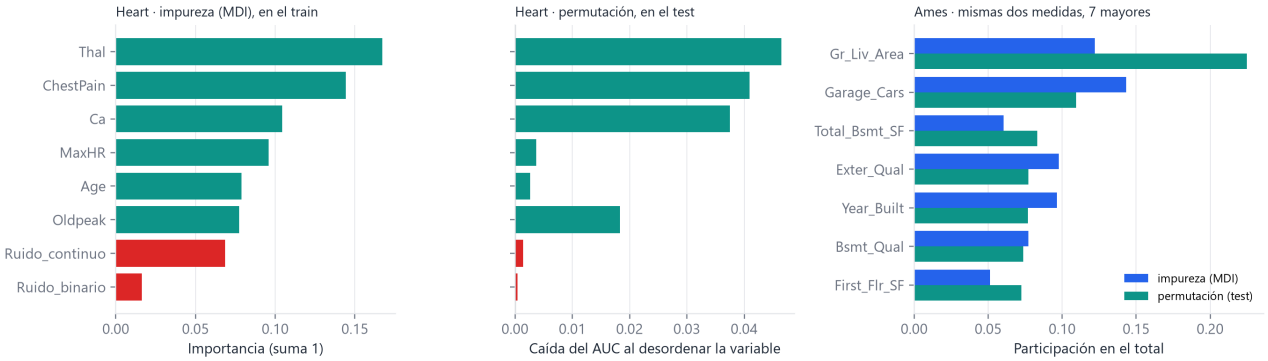
La importancia por **impureza** (MDI) suma, para cada variable, la reducción de Gini de sus cortes ponderada por la fracción de datos que llega al nodo. Se calcula con el train y premia a las variables con muchos cortes posibles. La de **permutación** desordena una variable en datos no vistos y mide cuánto empeora la métrica.

Ejemplo resuelto · importancia por impureza a mano

N = 80 (40 de cada clase). Raíz (Gini 0.5), corte por A: izquierda 30 (27/3, Gini 0.18), derecha 50 (13/37, Gini 0.3848). El nodo derecho se corta por B: 20 (11/9, Gini 0.495) y 30 (2/28, Gini 0.1244).

Corte	ΔGini del nodo	Peso N_t/N	Aporte
A (raíz)	0.5 − (30·0.18 + 50·0.3848)/80 = 0.5 − 0.308 = 0.192	80/80 = 1	0.192
B (nodo derecho)	0.3848 − (20·0.495 + 30·0.1244)/50 = 0.3848 − 0.2727 = 0.1121	50/80 = 0.625	0.0701

Total 0.2621 → importancia normalizada: A = 0.192/0.2621 = **0.733** y B = **0.267**. Permutación: si el AUC base es 0.88 y al desordenar Edad baja a 0.85, Colesterol a 0.879 y una columna de ruido a 0.881, las importancias son 0.03, 0.001 y −0.001.



Pista para la Parte D del notebook (D1) y el Bloque 8 del cuaderno

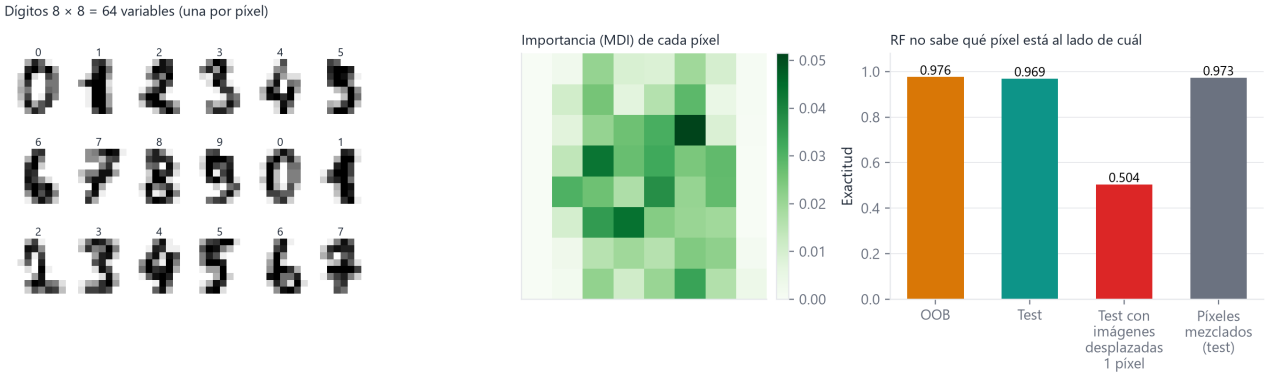
Agrega columnas de ruido a Heart y compara las dos importancias: el ruido continuo suele colarse por impureza y desaparecer por permutación. Recuerda que ninguna importancia es causalidad.

7 Límites: extrapolación, imágenes y modelos estables

No extrapola. Cada predicción de regresión es un promedio de valores de hojas, así que queda dentro del rango de entrenamiento. **Imágenes:** cada píxel es una columna; el bosque clasifica bien los dígitos (exactitud 0.969), pero no usa qué píxel está al lado de cuál. **Modelos estables:** el bagging ayuda donde el modelo base cambia mucho con los datos (árboles), no en una regresión logística.

Ejemplo resuelto · extrapolar con un bosque

Entrenas con casas de 100 a 500 millones. Llega una casa que debería valer 700. Un bosque promedia valores de hojas de entrenamiento: su predicción nunca supera ≈ 500. Una regresión lineal sí puede pasar de 500, bien o mal. En Ames, entrenado con casas < 1.800 pies² el bosque no pasó de ≈ 158 mil USD en las de > 2.400 pies² (RMSE 208 mil) y la lineal llegó a predecir ≈ 647 mil (RMSE 120 mil).

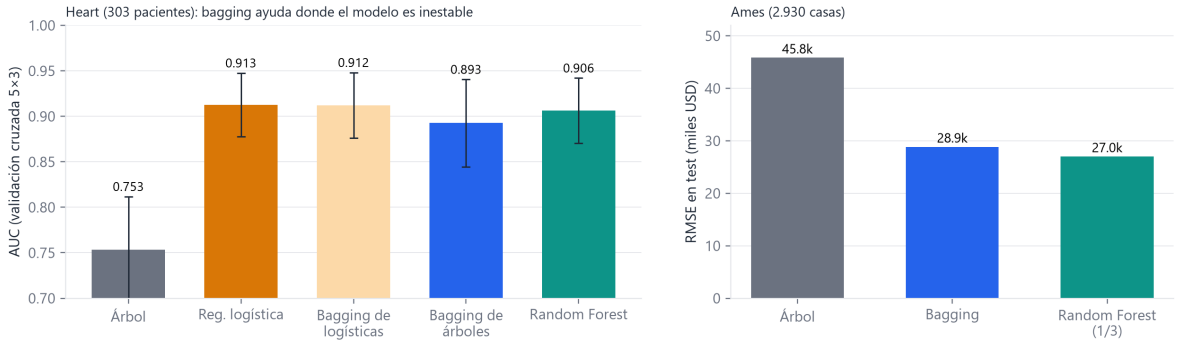


Pista para la Parte D del notebook (D2 a D4) y el Bloque 9 del cuaderno

En D3 mezcla los píxeles al azar (el bosque rinde igual: 0.973) y desplaza la imagen un píxel (cae a 0.504). Es la mejor demostración de qué «ve» un bosque.

8 Cómo elegir y comparar

Si lo que domina tu problema es…	Elige	Porque
Interpretar cada decisión	Un árbol (o regresión)	se lee de punta a punta
Datos tabulares con no linealidades, sin tiempo para ajustar mucho	Random Forest	buen desempeño con poco ajuste, sin escalar
Relaciones casi lineales, pocos datos	Regresión logística / lineal	con 303 pacientes igualó o superó al bosque (AUC 0.913)
Mejor error posible y puedes afinar	Boosting (Clase 36)	reduce sesgo en secuencia
Valores fuera del rango de entrenamiento	Un modelo que extrapole	los árboles se aplanan



Heart (CV 5×3): árbol 0.753 · logística 0.913 · bagging de logísticas 0.912 · bagging de árboles 0.893 · Random Forest 0.906, con desvíos ≈ 0.04: salvo el árbol, no hay diferencias claras. Ames (test): árbol 45.8k · bagging 28.9k · Random Forest 27.0k. Con una sola partición de 76 pacientes no se declara un ganador.

9 Chuleta de código

Quiero…	Código
Bagging con árboles	BaggingClassifier(DecisionTreeClassifier(), n_estimators=120, oob_score=True, n_jobs=-1)
Random Forest	RandomForestClassifier(n_estimators=300, max_features='sqrt', oob_score=True, n_jobs=-1, random_state=42)
Random Forest de regresión	RandomForestRegressor(n_estimators=300, max_features=0.2, min_samples_leaf=2, n_jobs=-1)
Error OOB	rf.oob_score_ · rf.oob_decision_function_ · rf.oob_prediction_ (regresión)
Importancia correcta	permutation_importance(pipe, X_test, y_test, scoring='roc_auc', n_repeats=30)
Optimizar	RandomizedSearchCV(pipe, {'model__max_features': [...]}, n_iter=12, scoring='roc_auc', cv=cv)

10 Errores comunes

Error	Por qué está mal	Qué hacer
Llamar «Random Forest» a max_features=1.0	Con todas las variables es un bagging	Probar max_features < 1
Usar solo más árboles para mejorar	El piso $\rho\sigma^2$ no baja	Bajar ρ (max_features) o cambiar de modelo
Fiarse de la impureza con ruido o categóricas largas	Premia variables con muchos cortes	Permutación sobre el test
Declarar ganador con 76 pacientes	1 paciente = 1.3 puntos de exactitud	CV repetida y media ± desvío
Buscar hiperparámetros contra el test	El test deja de ser honesto	CV u OOB; test al final
Esperar que el bosque extrapole	Promedia hojas del rango visto	Otro modelo o modelar la tendencia

Lo que debes recordar

- Orden sugerido: estos apuntes → Bloques 1–4 del cuaderno → Partes A y B del notebook → Bloques 5–7 → Parte C → Bloques 8–9 → Parte D → ejercicio de cierre.
- Si te trabas más de 10 minutos, mira la solución, entiéndela y vuelve a hacerlo sin mirar.